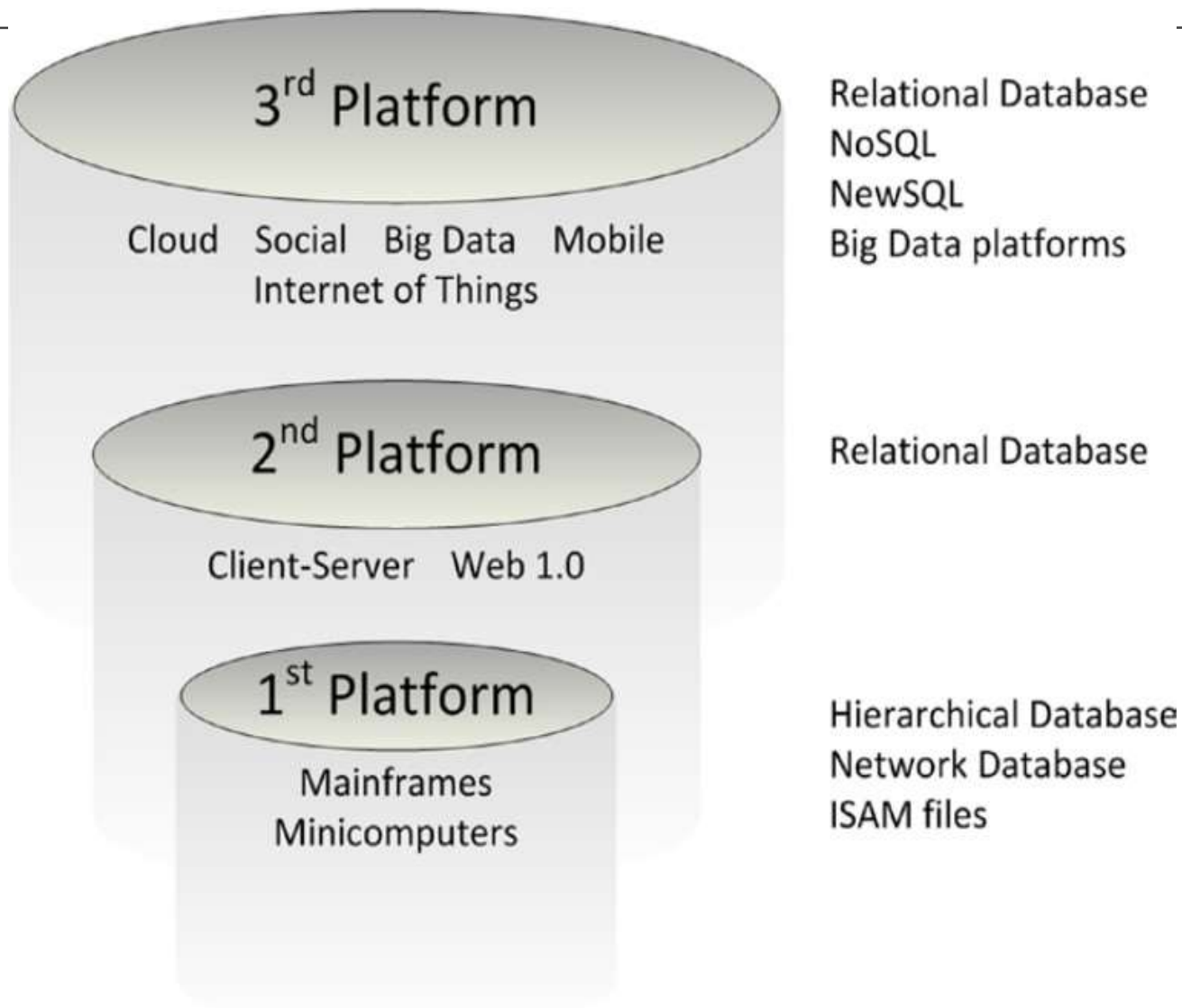

Advanced Database

NoSQL

Three Waves of Database Technology



NoSQL!

NoSQL databases are currently a hot topic in some parts of computing, with over a hundred different NoSQL databases.

HOW TO WRITE A CV



Leverage the NoSQL boom

RDBMS Characteristics

- Data stored in columns and tables
- Relationships represented by data
- Data Manipulation Language
- Data Definition Language
- Transactions
- Abstraction from physical layer
- Applications specify what, not how
- Physical layer can change without modifying applications
 - Create indexes to support queries
 - In Memory databases

Transactions – ACID Properties

- Atomic – All of the work in a transaction completes (commit) or none of it completes
 - a transaction to transfer funds from one account to another involves making a withdrawal operation from the first account and a deposit operation on the second. If the deposit operation failed, you don't want the withdrawal operation to happen either.
- Consistent – A transaction transforms the database from one consistent state to another consistent state. Consistency is defined in terms of constraints.
 - a database tracking a checking account may only allow unique check numbers to exist for each transaction
- Isolated – The results of any changes made during a transaction are not visible until the transaction has committed.
 - a teller looking up a balance must be isolated from a concurrent transaction involving a withdrawal from the same account. Only when the withdrawal transaction commits successfully and the teller looks at the balance again will the new balance be reported.
- Durable – The results of a committed transaction survive failures
 - A system crash or any other failure must not be allowed to lose the results of a transaction or the contents of the database. Durability is often achieved through separate transaction logs that can "re-create" all transactions from some picked point in time (like a backup).

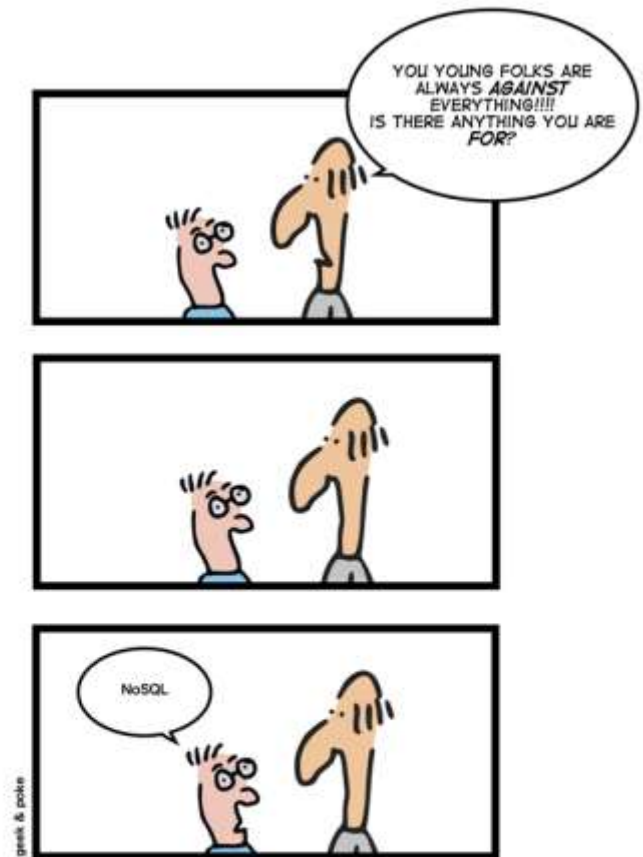
No SQL?

- NoSQL stands for:
 - No Relational
 - No RDBMS
 - Not Only SQL
- NoSQL is an umbrella term for all databases and data stores that don't follow the RDBMS principles
 - A class of products
 - A collection of several (related) concepts about data storage and manipulation
 - Often related to large data sets

NoSQL Definition

From www.nosql-database.org:

Next Generation Databases mostly addressing some of the points: being non-relational, distributed, open-source and horizontal scalable. The original intention has been modern web-scale databases. The movement began early 2009 and is growing rapidly. Often more characteristics apply as: schema-free, easy replication support, simple API, eventually consistent / BASE (not ACID), a huge data amount, and more.



NoSQL and Big Data

- NoSQL comes from Internet, thus it is often related to the “big data” concept
- How much big are “big data”?
 - Over few terabytes Enough to start spanning multiple storage units
- Challenges
 - Efficiently storing and accessing large amounts of data is difficult, even more considering fault tolerance and backups
 - Manipulating large data sets involves running immensely parallel processes
 - Managing continuously *evolving schema* and metadata for *semi-structured and un-structured* data is difficult

How did we get here?

- Explosion of social media sites (Facebook, Twitter) with large data needs
- Rise of cloud-based solutions such as Amazon S3 (simple storage solution)
- Just as moving to dynamically-typed languages (Python, Ruby, Groovy), a shift to dynamically-typed data with frequent schema changes
- Open-source community

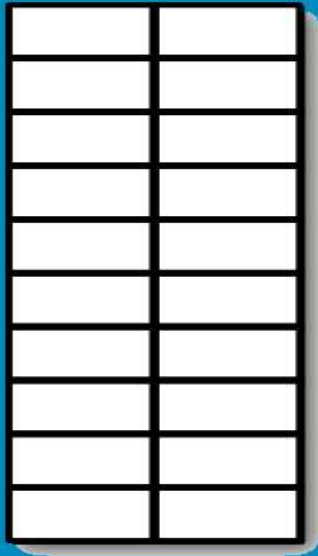
Why are RDBMS not suitable for Big Data

- The context is Internet
- RDBMSs assume that data are
 - Dense
 - Largely uniform (structured data)
- Data coming from Internet are
 - Massive and sparse
 - Semi-structured or unstructured
- With massive sparse data sets, the typical storage mechanisms and access methods get stretched

NoSQL Distinguishing Characteristics

- Large data volumes
 - Google's "big data"
- Scalable replication and distribution
 - Potentially thousands of machines
 - Potentially distributed around the world
- Queries need to return answers quickly
- Mostly query, few updates
- Asynchronous Inserts & Updates
- Schema-less
- ACID transaction properties are not needed — BASE
- Open source development

Key-Value



Graph DB

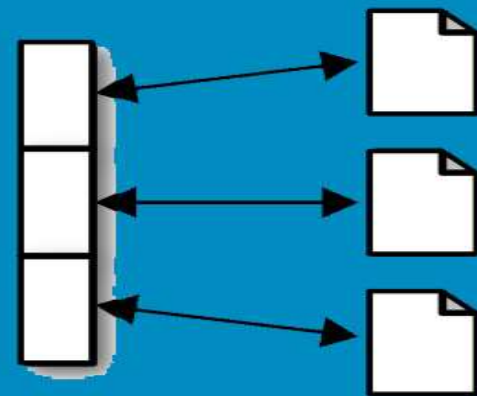


Column Family

A grid representing a Column Family database structure. The grid has 10 rows and 6 columns. Some cells contain the number 1, indicating data points across different columns.

				1	
1				1	
	1		1		
	1	1			
				1	
	1			1	
	1			1	
		1		1	
				1	

Document



NoSQL Database Types

Discussing NoSQL databases is complicated because there are a variety of types:

Sorted ordered Column Store

- Optimized for queries over large datasets, and store columns of data together, instead of rows

•Document databases:

- pair each key with a complex data structure known as a document.

•Key-Value Store :

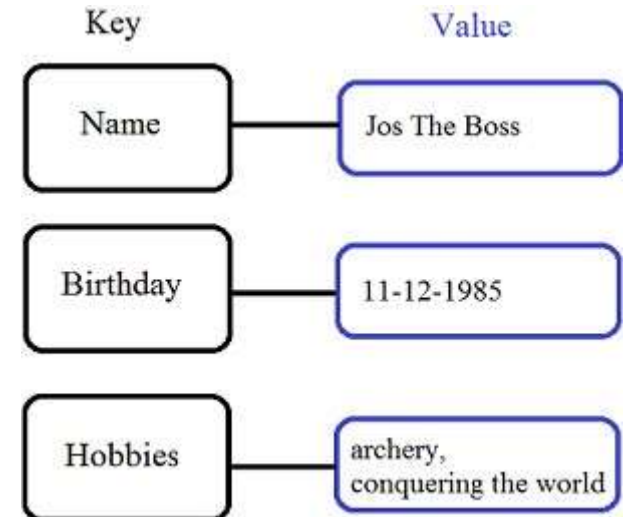
- are the simplest NoSQL databases. Every single item in the database is stored as an attribute name (or 'key'), together with its value.

•Graph Databases :

- are used to store information about networks of data, such as social connections.

Key/Value stores

- Store data in a schema-less way
- Store data as maps
 - HashMaps or associative arrays
 - Provide a very efficient average running time algorithm for accessing data
- Notable for:
 - Couchbase (Zynga, Vimeo, NAVTEQ, ...)
 - Redis (Craiglist, Instagram, StackOverflow, flickr, ...)
 - Amazon Dynamo (Amazon, Elsevier, IMDb, ...)
 - Apache Cassandra (Facebook, Digg, Reddit, Twitter,...)
 - Voldemort (LinkedIn, eBay, ...)
 - Riak (Github, Comcast, Mochi, ...)



Distributed Key-Value Data Stores

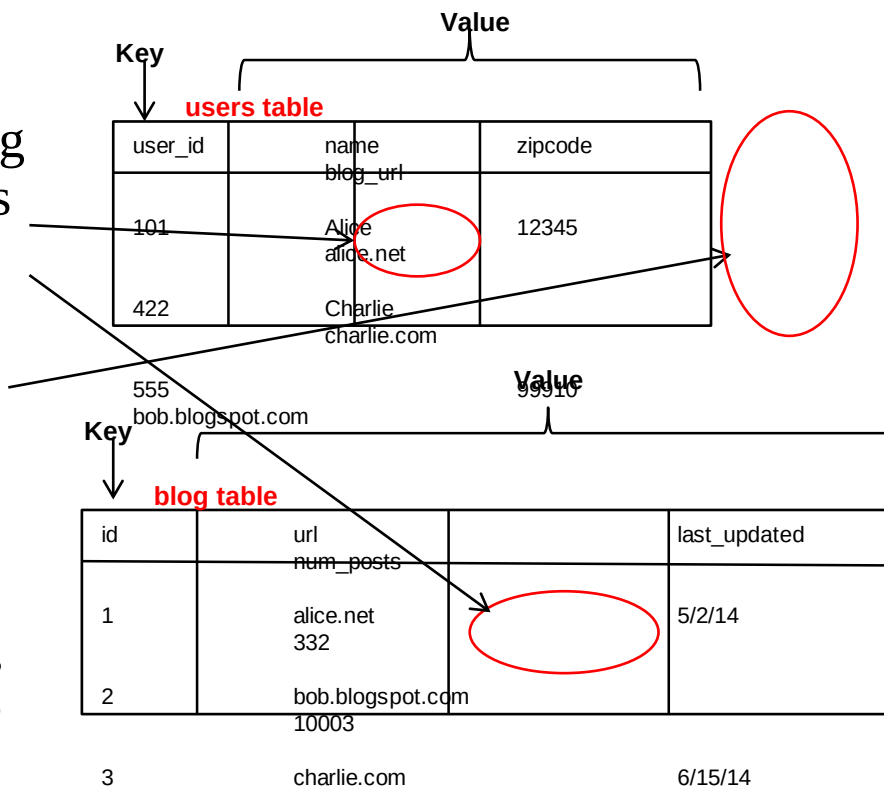
- Distributed key-value data storage systems allow key-value pairs to be stored (and retrieved on key) in a massively parallel system
 - E.g. Google BigTable, Yahoo! Sherpa/PNUTS, Amazon Dynamo, ..
- Partitioning, high availability etc completely transparent to application
- Sharding systems and key-value stores don't support many relational features
 - No join operations (except within partition)
 - No referential integrity constraints across partitions
 - etc.

The Key-value Abstraction

- It's a dictionary datastructure.
 - Insert, lookup, and delete by key
 - E.g., hash table, binary tree
- But distributed.
- Sound familiar? Remember Distributed Hash tables (DHT) in P2P systems?

Key-value/NoSQL Data Model

- Unstructured
- Columns Missing from some Rows
- No schema imposed
- No foreign keys, joins may not be supported



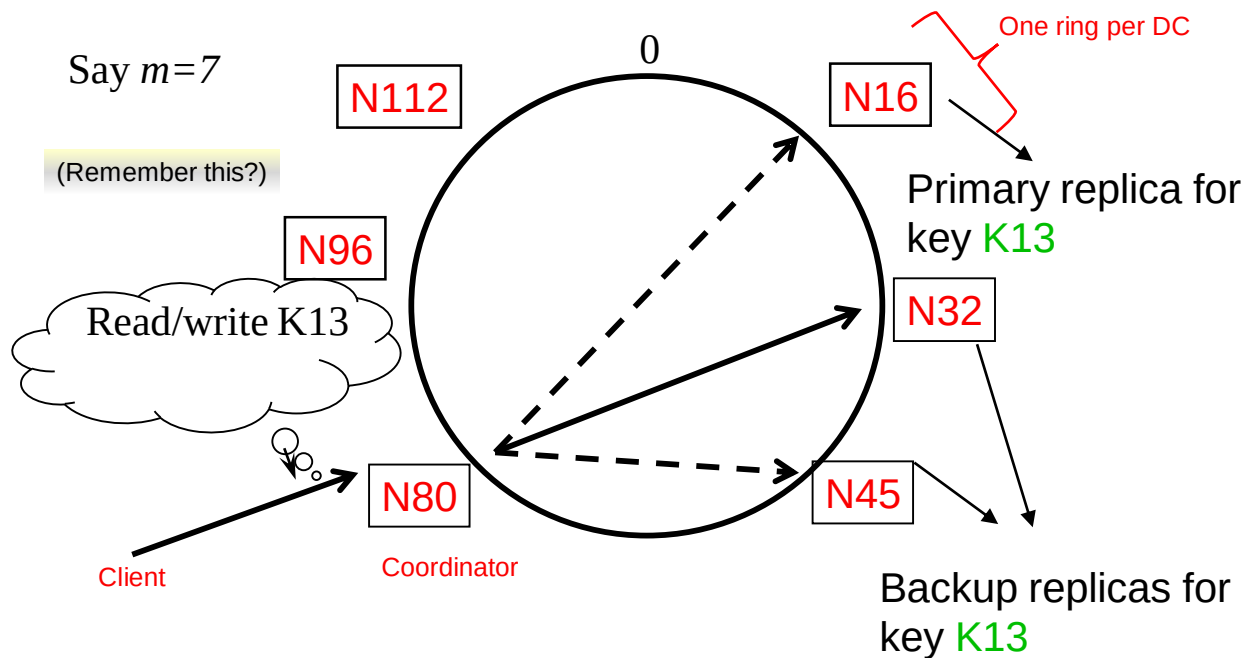
Cassandra

- A distributed key-value store
- Intended to run in a datacenter (and also across DCs)
- Originally designed at Facebook
- Open-sourced later, today an Apache project
- Some of the companies that use Cassandra in their production clusters
 - IBM, Adobe, HP, eBay, Ericsson, Symantec
 - Twitter, Spotify
 - PBS Kids
 - Netflix: uses Cassandra to keep track of current position in the video watching

Let's go Inside Cassandra:

Key -> Server Mapping

- How do you decide which server(s) a key-value resides on?



Cassandra uses a Ring-based DHT but without finger tables or routing
Key $\rightarrow$ server mapping is the "Partitioner"

Column-Oriented Stores

- Data are stored in a column-oriented way
 - Data efficiently stored
 - Avoids consuming space for storing nulls
 - Columns are grouped in column-families
 - Data isn't stored as a single table but is stored by column families
 - Unit of data is a set of key/value pairs
 - Identified by “row-key”
 - Ordered and sorted based on row-key
- Notable for:
 - Google's Bigtable (used in all Google's services)
 - HBase (Facebook, StumbleUpon, Hulu, Yahoo!, ...)

Sales			
Product	Customer	Date	Sale
Beer	Thomas	2011-11-25	2 GBP
Beer	Thomas	2011-11-25	2 GBP
Vodka	Thomas	2011-11-25	10 GBP
Whiskey	Christian	2011-11-25	5 GBP
Whiskey	Christian	2011-11-25	5 GBP
Vodka	Alexei	2011-11-25	10 GBP
Vodka	Alexei	2011-11-25	10 GBP

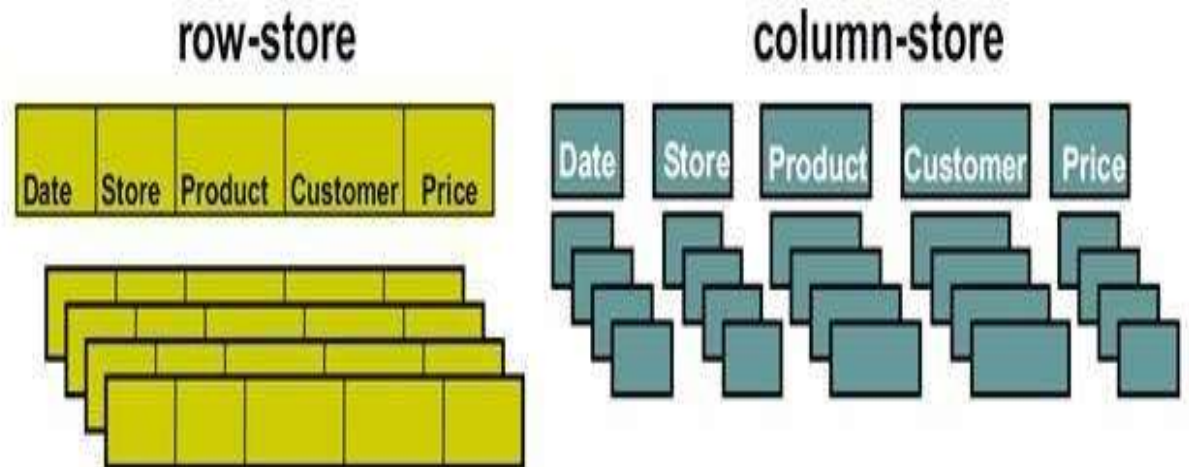
Product	
ID	Value
1	Beer
2	Beer
3	Vodka
4	Whiskey
5	Whiskey
6	Vodka
7	Vodka

Customer	
ID	Customer
1	Thomas
2	Thomas
3	Thomas
4	Christian
5	Christian
6	Alexei
7	Alexei



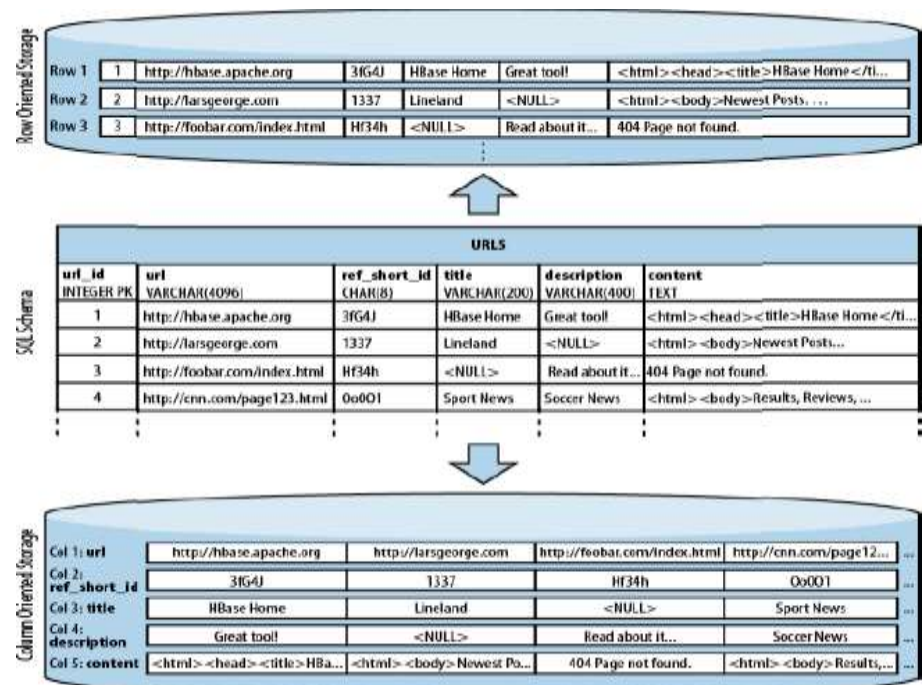
Data Model (1/7)

- ▶ Column-Oriented data model
- ▶ Similar to a **key/value** store, but the **value** can have multiple **attributes** (Columns).
- ▶ **Column**: a set of data **values** of a particular **type**.
- ▶ Store and process data by **column** instead of **row**.



Data Model (2/7)

- ▶ In many analytical databases queries, **few attributes** are needed.
- ▶ **Column values** are stored **contiguously** on disk: **reduces I/O**.



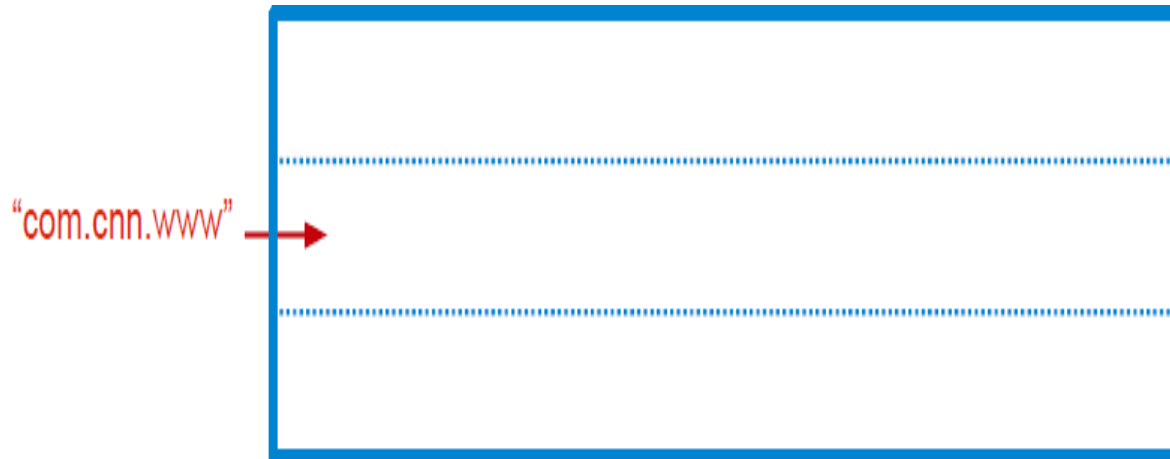
Data Model (3/7)

- ▶ Table
- ▶ Distributed multi-dimensional sparse `map`



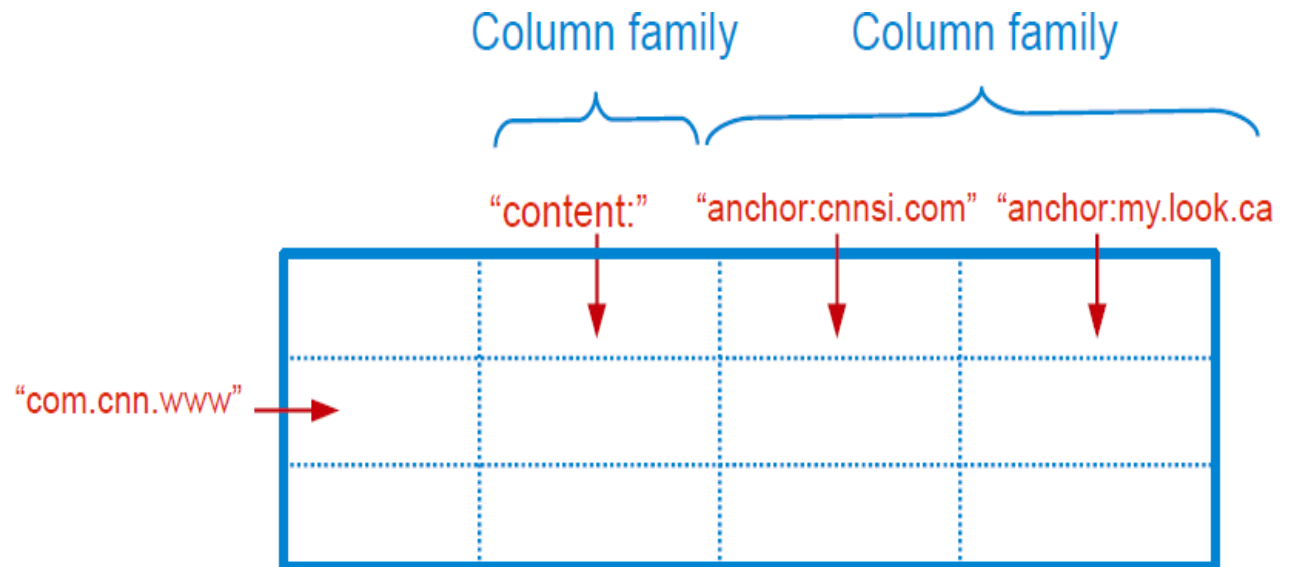
Data Model (4/7)

- ▶ Rows
- ▶ Every read or write in a **row** is **atomic**.
- ▶ Rows sorted in **lexicographical** order.



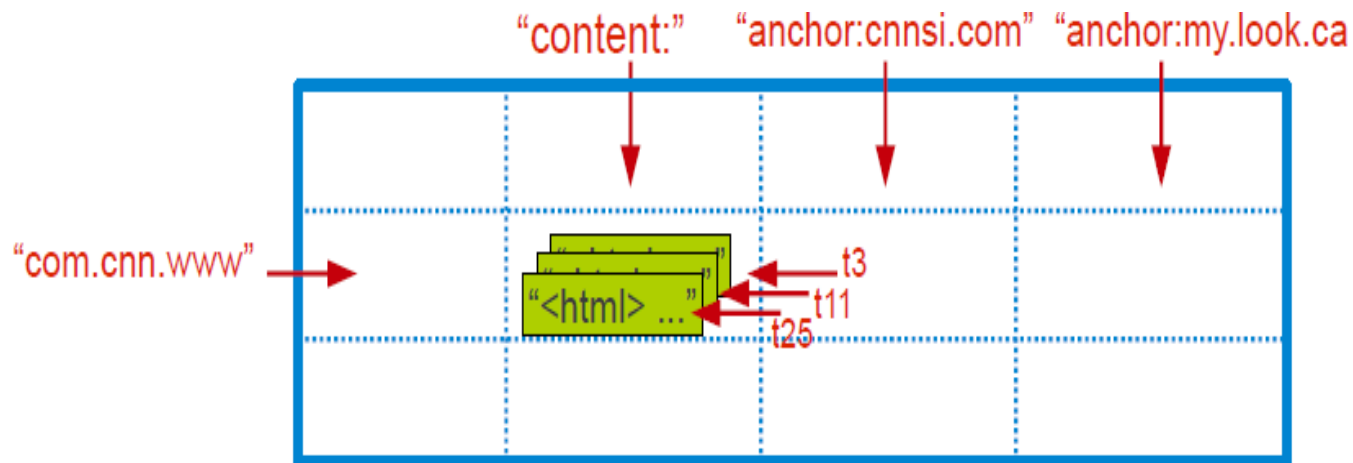
Data Model (5/7)

- ▶ Column
- ▶ The **basic unit** of data access.
- ▶ **Column families**: group of (the same type) column keys.
- ▶ Column key naming: **family:qualifier**



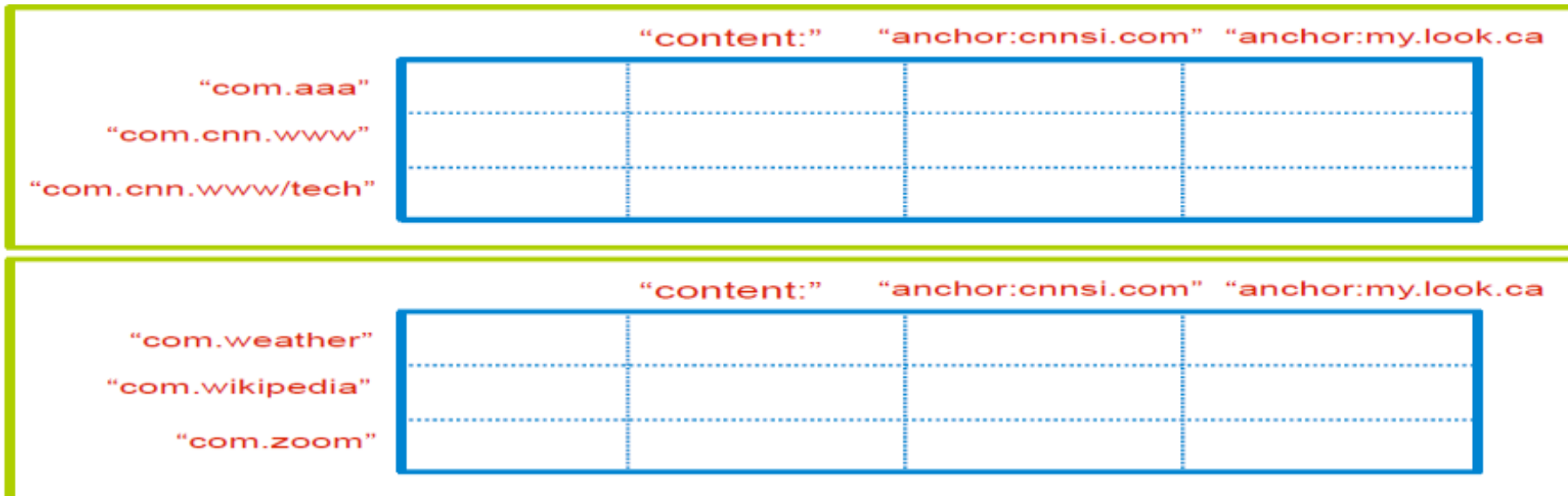
Data Model (6/7)

- ▶ Timestamp
- ▶ Each column value may contain multiple versions.



Data Model (7/7)

- ▶ **Tablet**: contiguous ranges of rows stored together.
- ▶ Tablets are **split** by the system when they become too large.
- ▶ Each **tablet** is served by exactly one **tablet**



Document Databases (Document Store)

- Documents

- Loosely structured sets of key/value pairs in documents, e.g., XML, JSON, BSON
- Encapsulate and encode data in some standard formats or encodings
- Are addressed in the database via a **unique key**
- Documents are treated as a whole, avoiding splitting a document into its constituent name/value pairs

- Allow documents retrieving by keys or contents

- Notable for:

- MongoDB (used in FourSquare, Github, and more)
- CouchDB (used in Apple, BBC, Canonical, Cern, and more)

Document Databases (Document Store)

- The central concept is the notion of a "document" which corresponds to a row in RDBMS.
- A document comes in some standard formats like JSON (BSON).
- The database offers an API or query language that retrieves documents based on their contents.
- Documents are schema free, i.e., different documents can have structures and schema that differ from one another. (An RDBMS requires that each row contain the same columns.)

Document Databases, JSON

```
{
  _id: ObjectId("51156a1e056d6f966f268f81"),
  type: "Article",
  author: "Derick Rethans",
  title: "Introduction to Document Databases with MongoDB",
  date: ISODate("2013-04-24T16:26:31.911Z"),
  body: "This arti..."
},
{
  _id: ObjectId("51156a1e056d6f966f268f82"),
  type: "Book",
  author: "Derick Rethans",
  title: "php|architect's Guide to Date and Time Programming with
PHP",
  isbn: "978-0-9738621-5-7"
}
```

MongoDB: Typical Query

Query all employee names with salary greater than 18000 sorted in ascending order

```
db.employee.find({salary:{$gt:18000}, {name:1}}).sort({salary:1})
```

Collection

Condition

Projection

Modifier

{salary:25000, ...}
{salary:10000, ...}
{salary:20000, ...}
{salary:2000, ...}
{salary:30000, ...}
{salary:21000, ...}
{salary:5000, ...}
{salary:50000, ...}

{salary:25000, ...}
{salary:20000, ...}
{salary:30000, ...}
{salary:21000, ...}
{salary:50000, ...}

{salary:20000, ...}
{salary:21000, ...}
{salary:25000, ...}
{salary:30000, ...}
{salary:50000, ...}

Insert

Insert a row entry for new employee Sally

```
db.employee.insert({  
    name: "sally",  
    salary: 15000,  
    designation: "MTS",  
    teams: [ "cluster-management" ]  
})`
```

Update

All employees with salary greater than 18000 get a designation of Manager

<i>Update Criteria</i>	<code>db.employee.update(</code>
	<code> {salary:{\$gt:18000}},</code>
<i>Update Action</i>	<code> {\$set: {designation: "Manager"}},</code>
<i>Update Option</i>	<code> {multi: true}</code>
	<code>)</code>

Multi-option allows multiple document update

Delete

Remove all employees who earn less than 10000

Remove Criteria

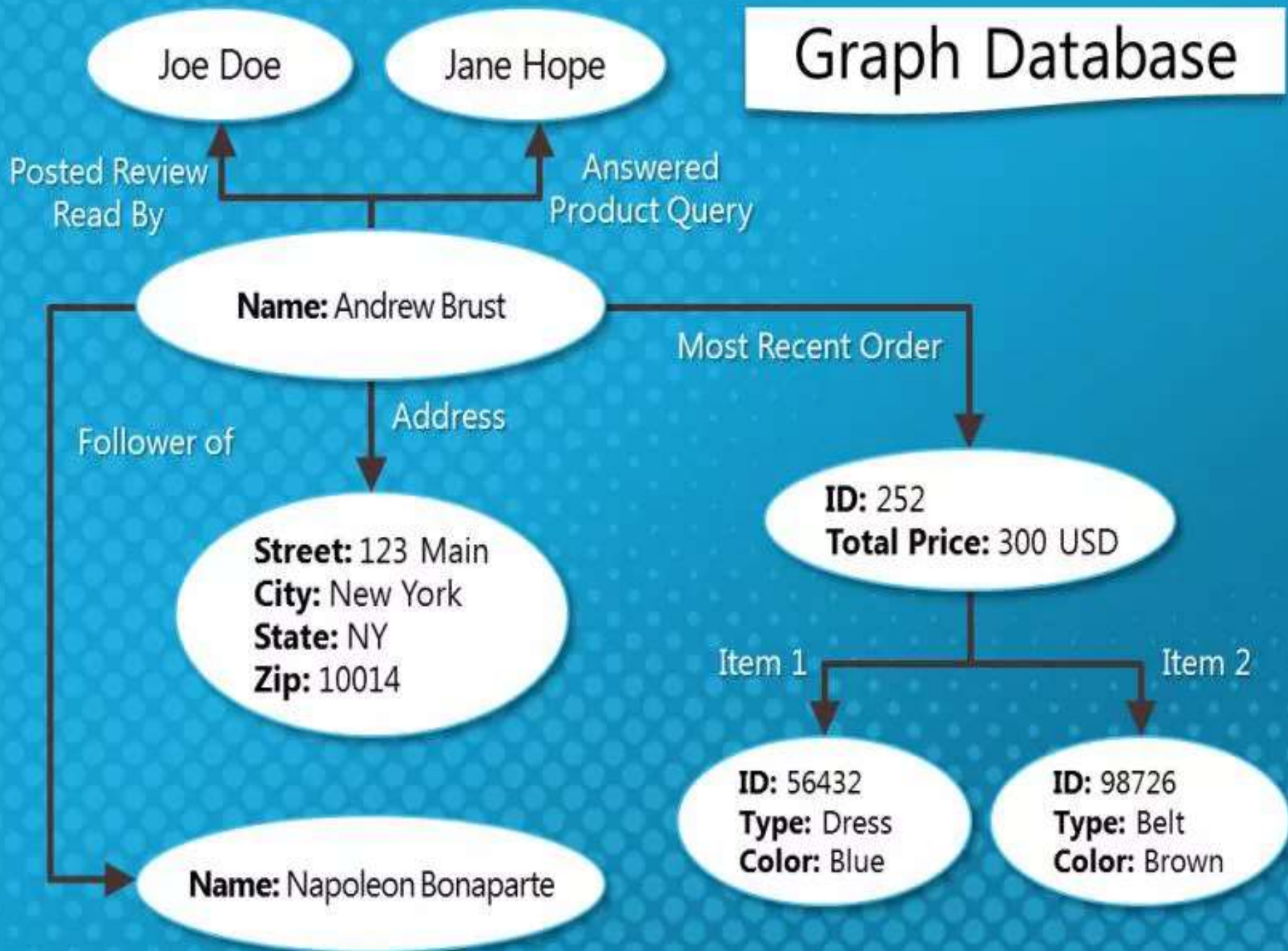
```
db.employee.remove(  
    {salary:{$lt:10000}},  
)
```

Can accept a flag to limit the number of documents removed

Graph Databases

- Graph-oriented
- Everything is stored as an edge, a node or an attribute.
- Each node and edge can have any number of attributes.
- Both the nodes and edges can be labelled.
- Labels can be used to narrow searches.

Graph Database



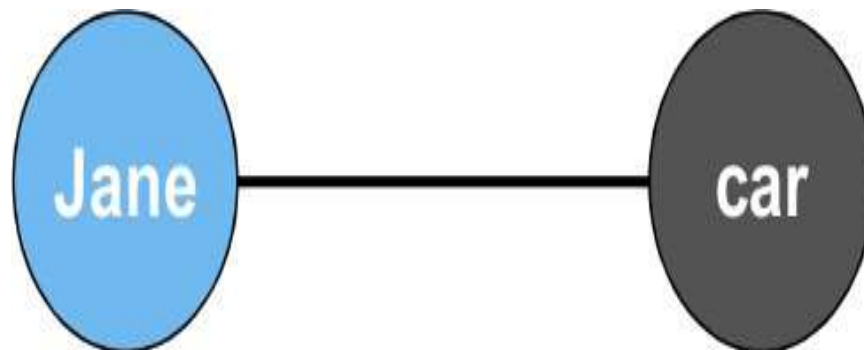
Neo4j

- ▶ A graph database
- ▶ The relationships between data is equally important as the data itself
- ▶ Cypher: a declarative query language similar to SQL, but optimized for graphs
- ▶ CAP: strong consistency and availability



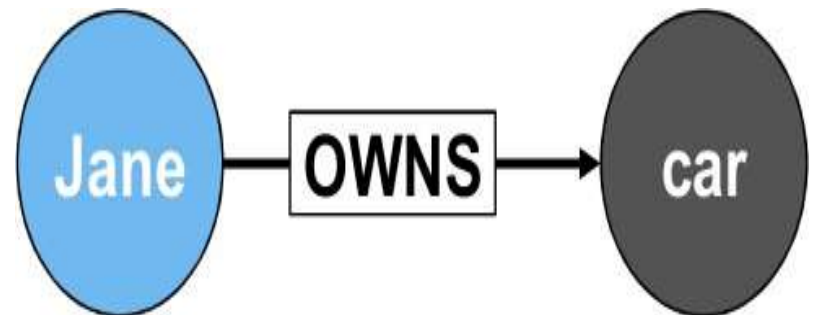
Data Model (1/4)

- ▶ Node (Vertex)
 - The main data element from which graphs are constructed.
 - A waypoint along a traversal route



Data Model (2/4)

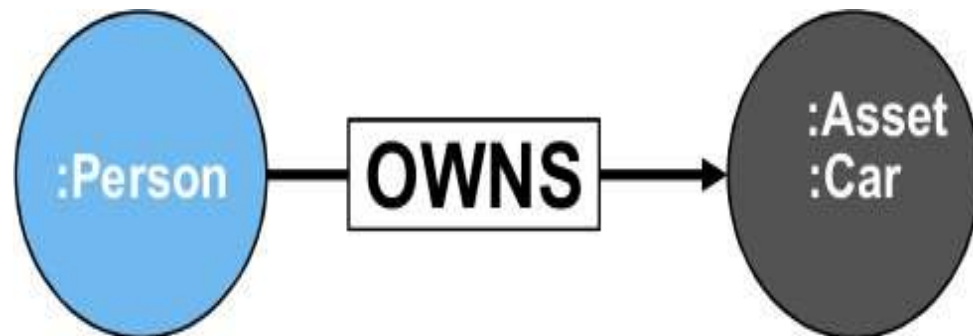
- ▶ Relationship (Edge)
- ▶ May contain
 - Direction
 - Metadata, e.g., weight or relationship type



Data Model (3/4)

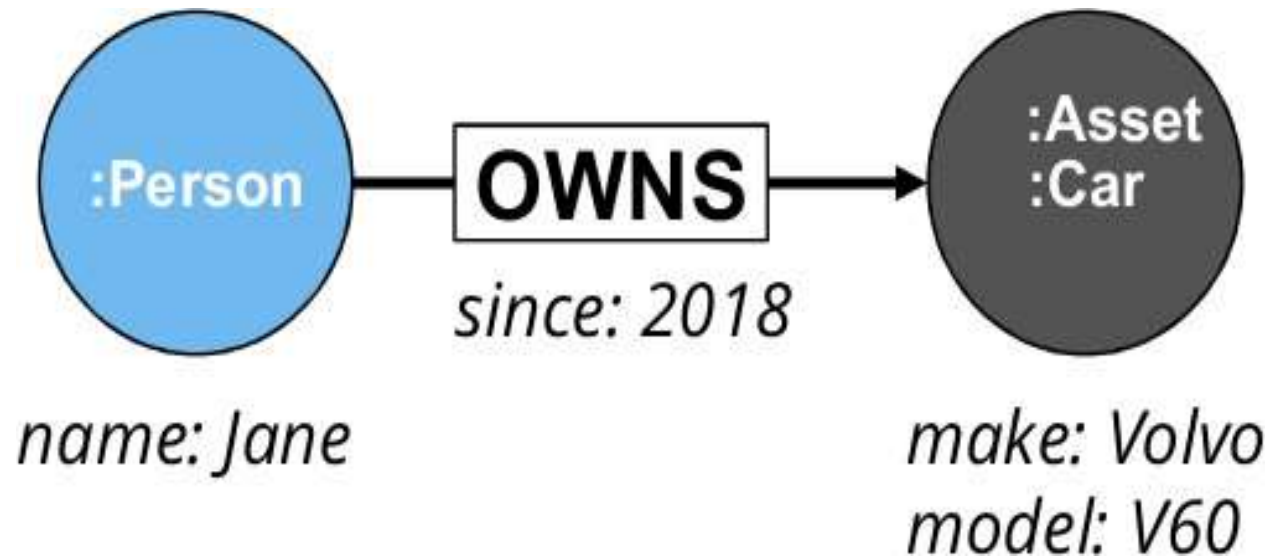
► Label

- Define node category (optional)
- Can have more than one

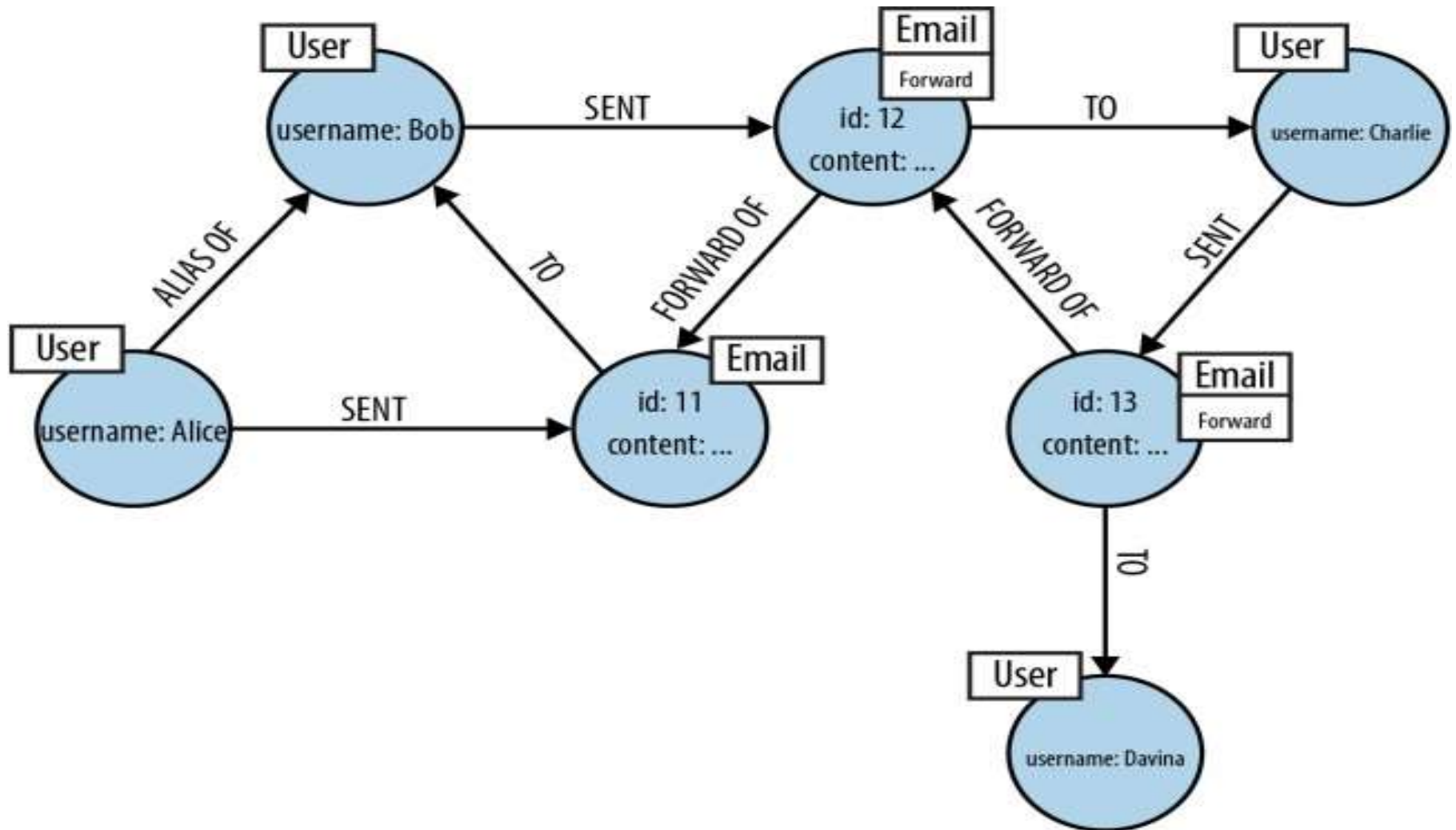


► Properties

- **Enrich** a node or relationship



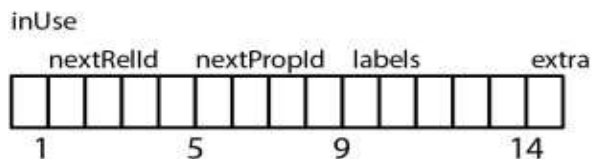
Example



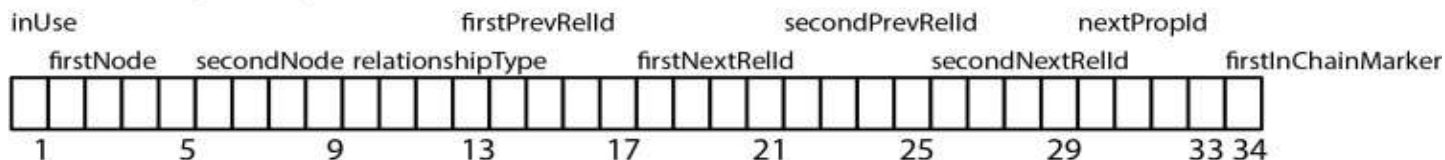
How a Graph is Physically Stored in Neo4j? (1/2)

- ▶ Neo4j stores graph data in a number of different **store files**.
- ▶ Each **store file** contains the data for a **specific part** of the graph.
 - Separate stores for **nodes**, **relationships**, **labels**, and **properties**.
- ▶ The division of storage responsibilities **facilitates performant graph traversals**.

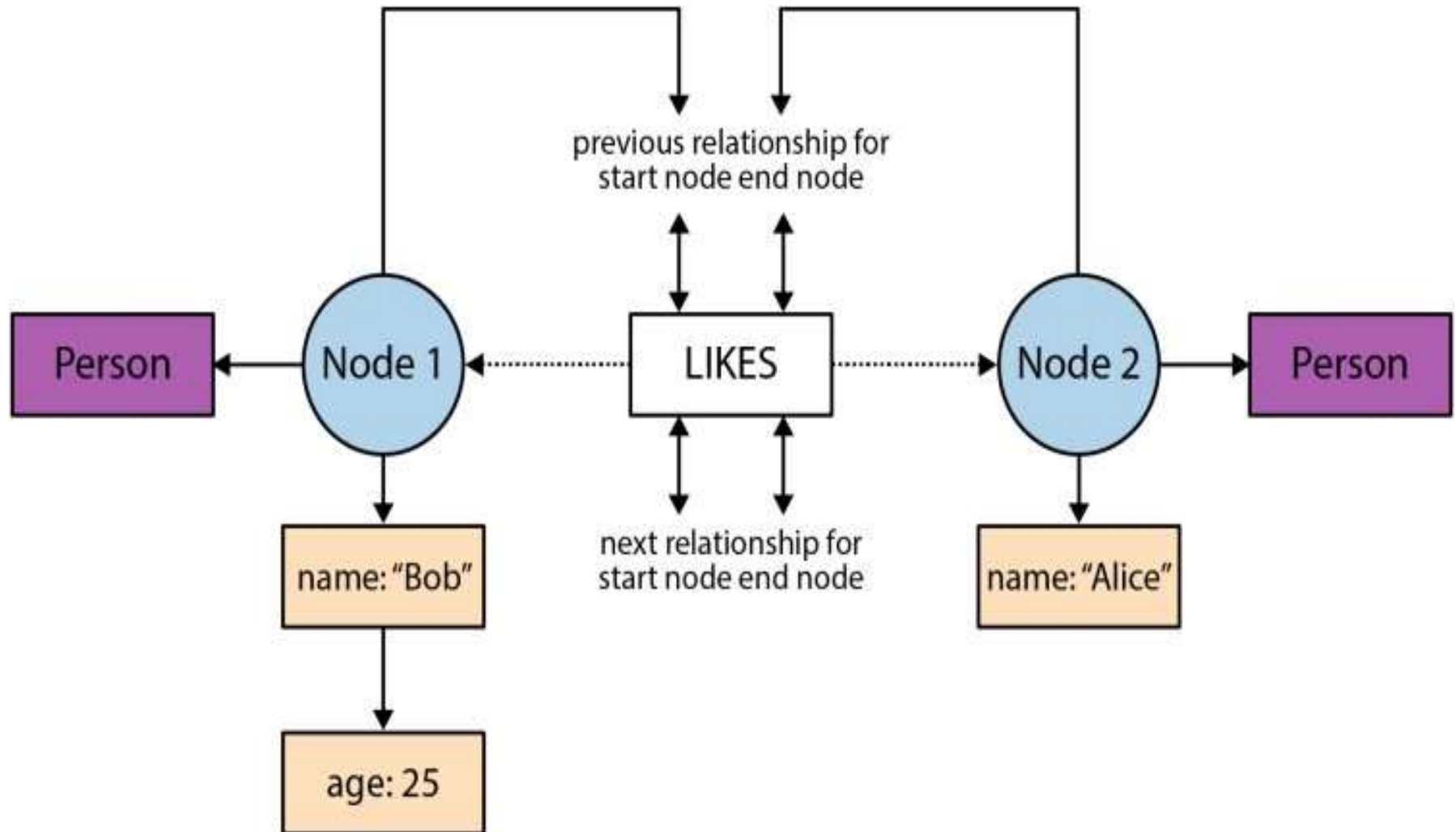
Node (15 bytes)



Relationship (34 bytes)



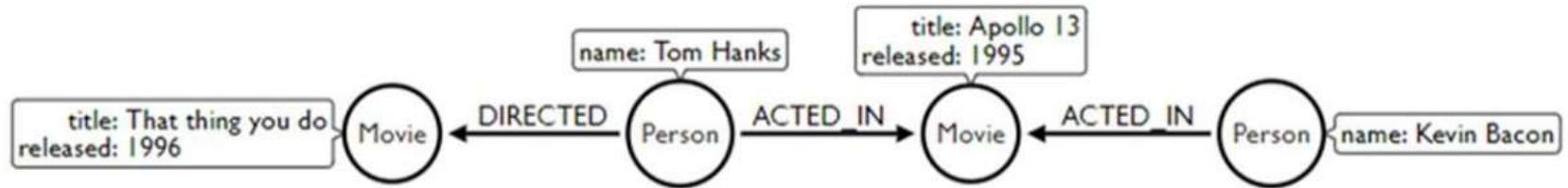
How a Graph is Physically Stored in Neo4j? (2/2)



What is Cypher?

- ▶ **Declarative** query language
- ▶ **()**: Nodes
- ▶ **[]**: Relationships
- ▶ **{}**: Properties

Cypher Example (1/4)

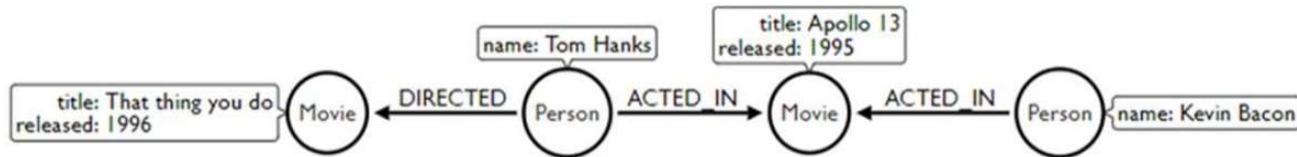


```
// Match all nodes MATCH (n)  
RETURN n;
```

```
// Match all nodes with a Person label  
MATCH (n:Person)  
RETURN n;
```

```
// Match all nodes with a Person label and  
property name is 'Tom Hanks' MATCH  
(n:Person {name: 'Tom Hanks'})  
RETURN n;
```

Cypher Example (2/4)

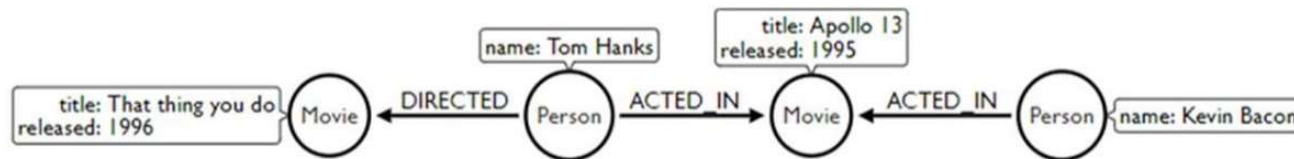


```
// Return nodes with label Person and name property equals 'Tom Hanks'  
MATCH (p:Person)  
WHERE p.name = 'Tom Hanks'  
RETURN p;
```

```
// Return nodes with label Movie, released property is between  
1991 and 1999  
MATCH (m:Movie)  
WHERE m.released > 1990 AND m.released < 2000  
RETURN m;
```

```
// Find all the movies Tom Hanks acted in  
MATCH (:Person {name:'Tom Hanks'})-[:ACTED_IN]-  
>(m:Movie) RETURN m.title;
```

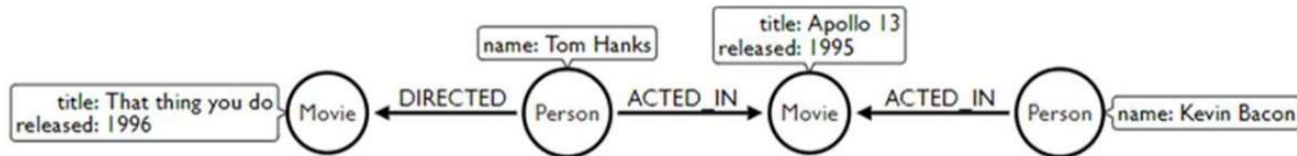

Cypher Example (3/4)



```
// Find all the movies Tom Hanks directed and order by  
latest movie MATCH (:Person {name:'Tom Hanks'})-  
[:DIRECTED]->(m:Movie)  
RETURN m.title, m.release ORDER BY m.release DESC;
```

```
// Find all of the co-actors Tom Hanks has ever worked with  
MATCH (:Person {name:'Tom Hanks'})-->(:Movie)<-  
[:ACTED_IN]-(coActor:Person) RETURN coActor.name;
```

Cypher Example (4/4)



```
// Find nodes with an ACTED_IN relationship
MATCH (p)-[:ACTED_IN]->()
RETURN p
```

```
// Find Person nodes with an ACTED_IN or DIRECTED_IN
relationship MATCH (p:Person)-[:ACTED_IN|DIRECTED]->()
RETURN p
```

```
// Find Person nodes who do not have an ACTED_IN
relationship MATCH (p:Person)
WHERE NOT (p)-[:ACTED_IN]->()
RETURN p
```

Dealing with Big Data and Scalability

- Issues with scaling up when the dataset is just too big
- RDBMS were not designed to be distributed
- Traditional DBMSs are best designed to run well on a “single” machine
 - Larger volumes of data/operations requires to upgrade the server with faster CPUs or more memory known as ‘scaling up’ or ‘Vertical scaling’
- NoSQL solutions are designed to run on clusters or multi-node database solutions
 - Larger volumes of data/operations requires to add more machines to the cluster, Known as ‘scaling out’ or ‘horizontal scaling’
 - Different approaches include:
 - Master-slave
 - Sharding (partitioning)

Scaling

- Master-Slave

- All writes are written to the master. All reads performed against the replicated slave databases
- Critical reads may be incorrect as writes may not have been propagated down
- Large data sets can pose problems as master needs to duplicate data to

- Sharding

- Any DB distributed across multiple machines needs to know in what machine a piece of data is stored or must be stored
- A sharding system makes this decision for each row, using its key

NoSQL, No ACID

- RDBMSs are based on ACID (Atomicity, Consistency, Isolation, and Durability) properties
- NoSQL
 - Does not give importance to ACID properties
 - In some cases completely ignores them
- In distributed parallel systems it is difficult/impossible to ensure ACID properties
 - Long-running transactions don't work because keeping resources blocked for a long time is not practical

BASE Transactions

- Acronym contrived to be the opposite of ACID
 - **B**asically **A**vailable,
 - **S**oft state,
 - **E**ventually **C**onsistent
- Characteristics
 - Weak consistency – stale data OK
 - Availability first
 - Best effort
 - Approximate answers OK
 - Aggressive (optimistic)
 - Simpler and faster

BASE Properties

▶ Basic Availability

- Possibilities of faults but not a fault of the whole system.

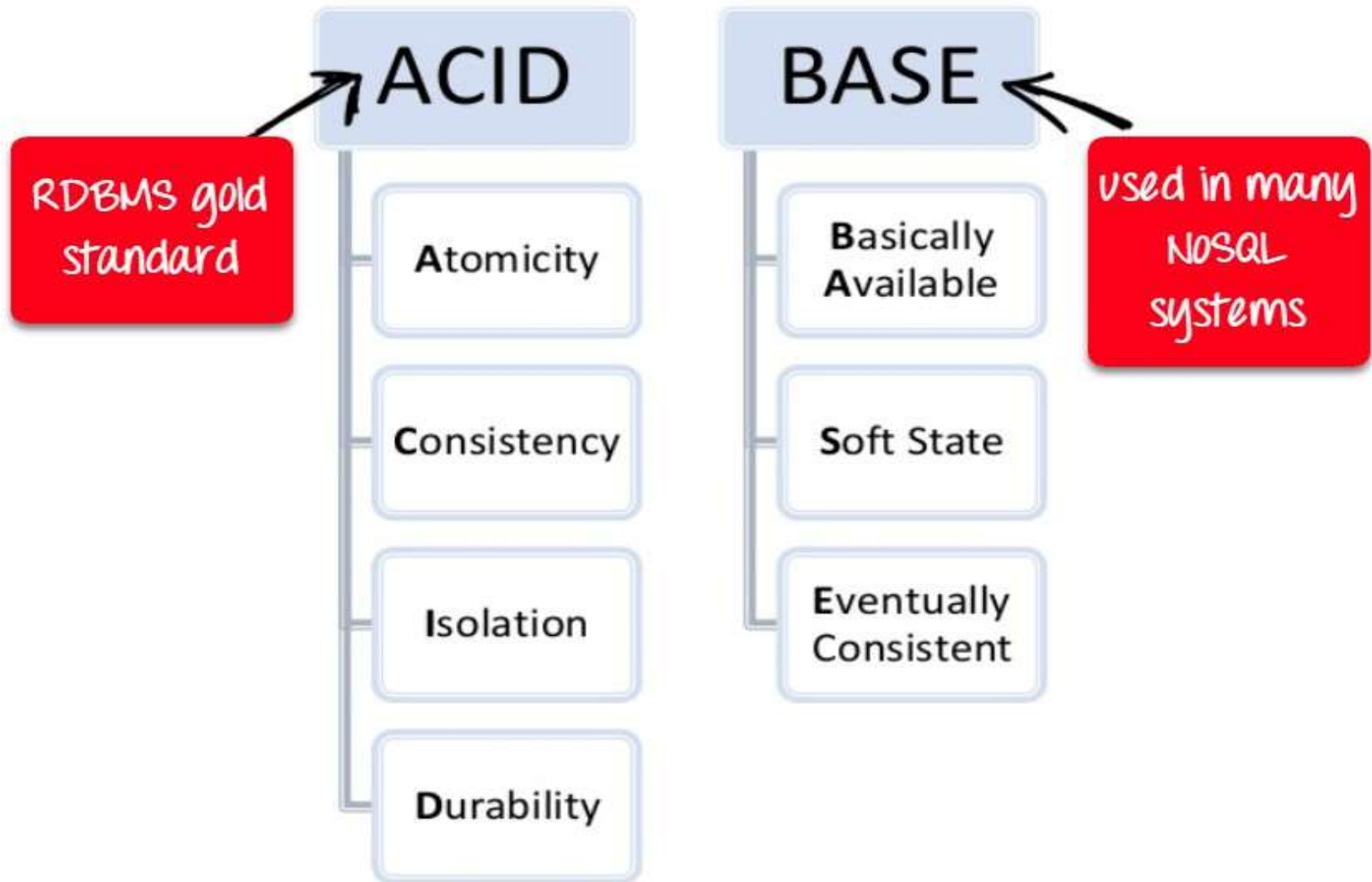
▶ Soft-state

- Copies of a data item may be inconsistent

▶ Eventually consistent

- Copies becomes consistent at some later time if there are no more updates to that data item

ACID vs. BASE



CAP Theorem

A congruent and logical way for assessing the problems involved in assuring ACID-like guarantees in distributed systems is provided by the CAP theorem

At most two of the following three can be maximized at one time

- Consistency
 - Each client has the same view of the data
- Availability
 - Each client can always read and write
- Partition tolerance
 - System works well across distributed physical networks

Visual Guide to NoSQL Systems

Availability:
Each client can
always read
and write.

A

Data Models

Relational (comparison)
Key-Value
Column-Oriented/Tabular
Document-Oriented

CA

RDBMSs
(MySQL,
Postgres,
etc)

Aster Data
Greenplum
Vertica

AP

Dynamo
Voldemort
Tokyo Cabinet
KAI

Cassandra
SimpleDB
CouchDB
Riak

Pick Two

C

Consistency:
All clients always
have the same view
of the data.

CP

BigTable
Hypertable
Hbase

MongoDB
Terrastore
Scalaris

Berkeley DB
MemcacheDB
Redis

P

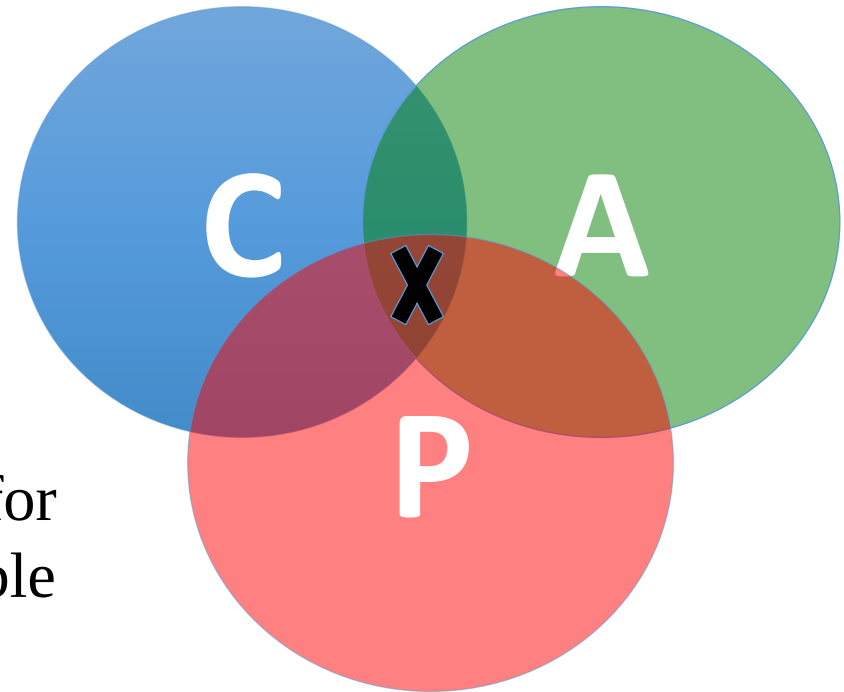
Partition Tolerance:
The system works
well despite physical
network partitions.

CAP Theorem: Two out of Three

- CAP theorem – At most two properties on three can be addressed
- The choices could be as follows:
 1. Availability is compromised but consistency and partition tolerance are preferred over it
 2. The system has little or no partition tolerance. Consistency and availability are preferred
 3. Consistency is compromised but systems are always available and can work when parts of it are partitioned

Consistency or Availability

- Consistency and Availability is not “binary” decision
- AP systems relax consistency in favor of availability – but are not inconsistent
- CP systems sacrifice availability for consistency- but are not unavailable
- This suggests both AP and CP systems can offer a degree of consistency, and availability, as well as partition tolerance

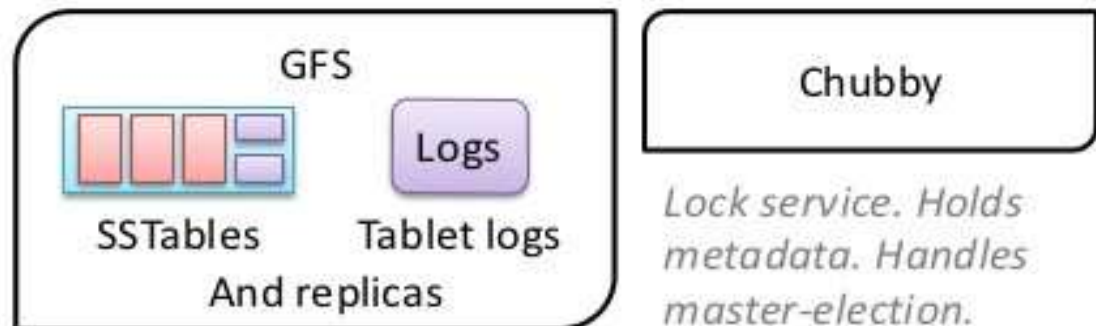
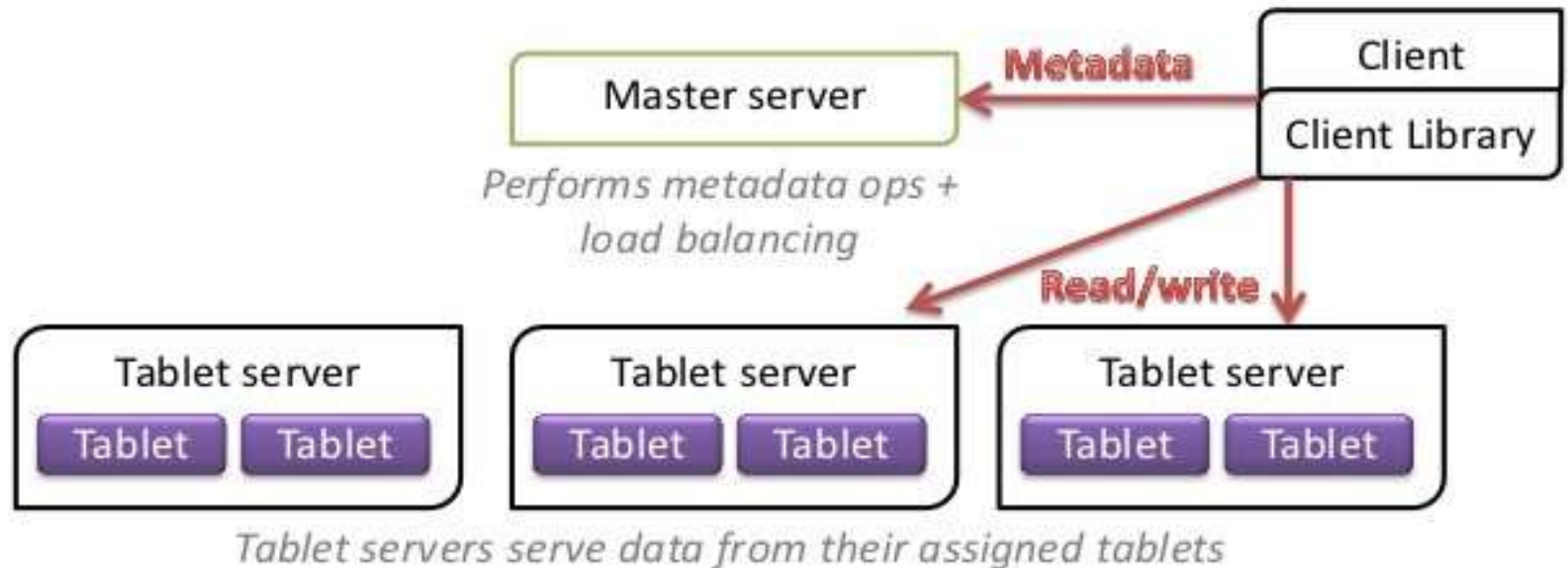


BigTable

- ▶ Lots of (semi-)structured data at Google.
 - URLs, per-user data, geographical locations, ...
- ▶ Distributed multi-level map
- ▶ CAP: strong consistency and partition tolerance

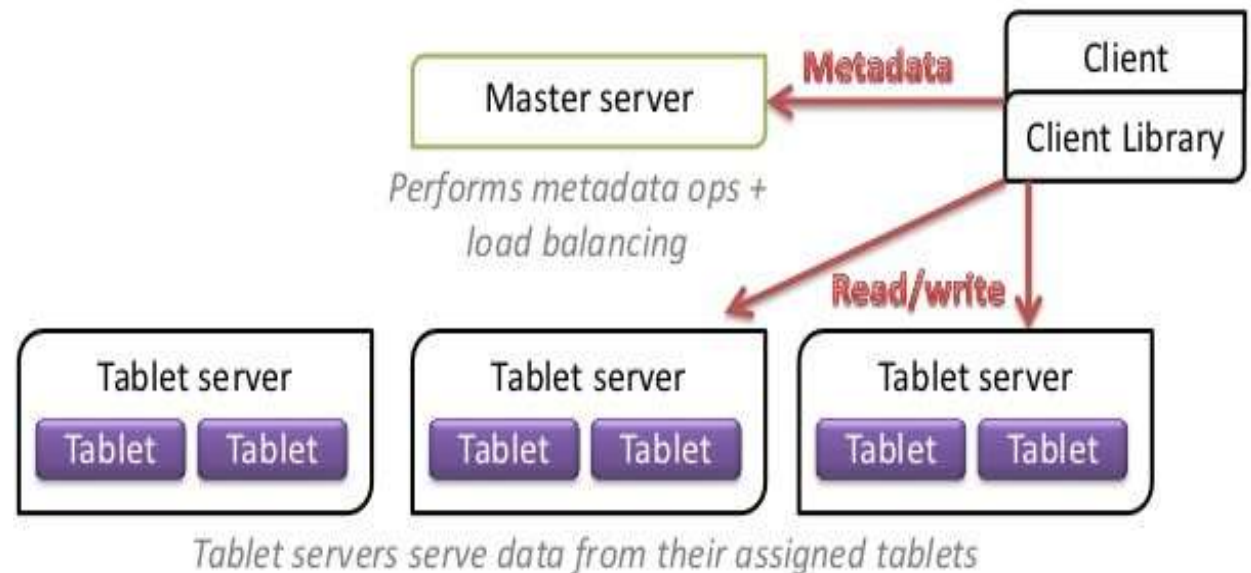


BigTable System Structure



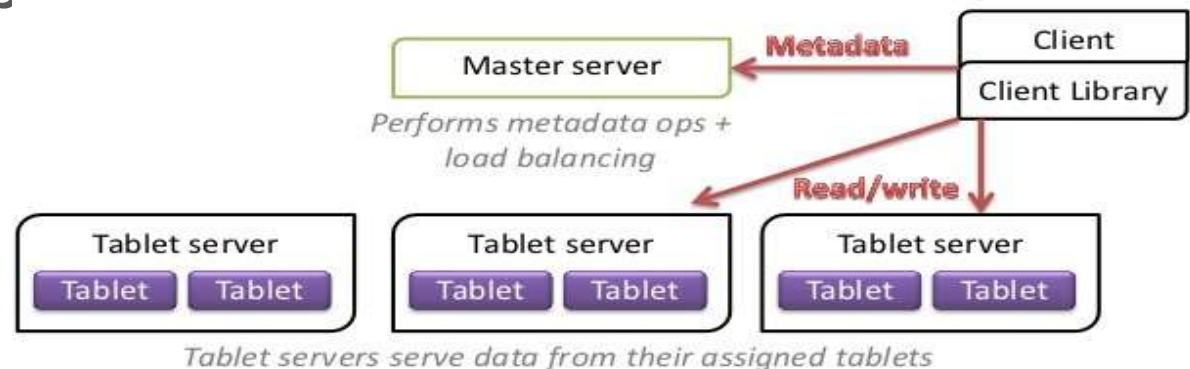
Main Components

- ▶ Master
- ▶ Tablet server
- ▶ Client library



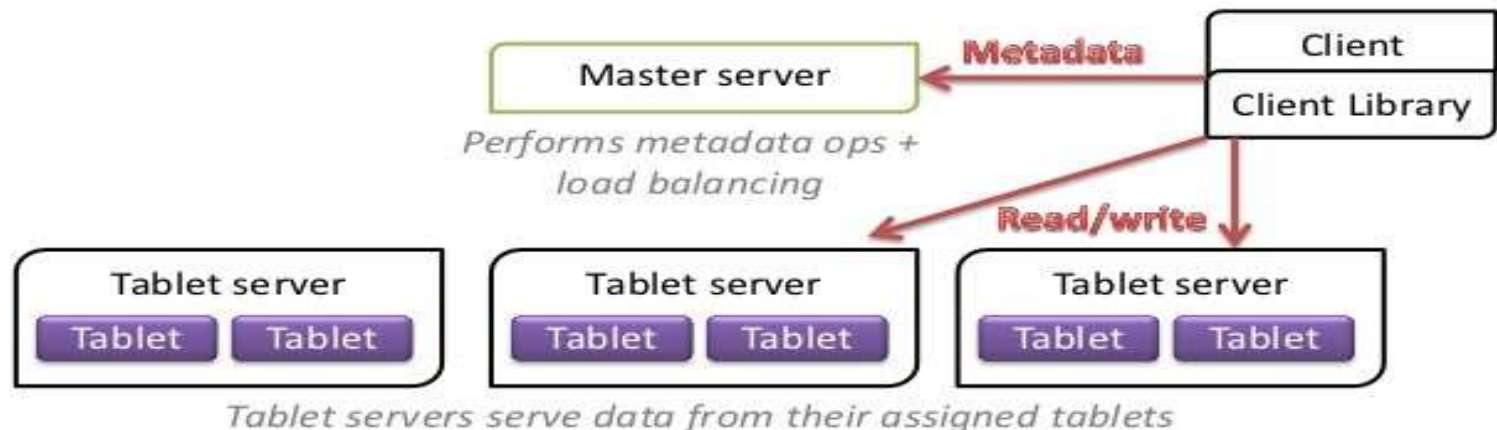
Master

- ▶ **Assigns tablets** to tablet server.
- ▶ **Balances** tablet server load.
- ▶ **Garbage collection** of unneeded files in GFS.
- ▶ Handles **schema changes**, e.g., table and column family creations



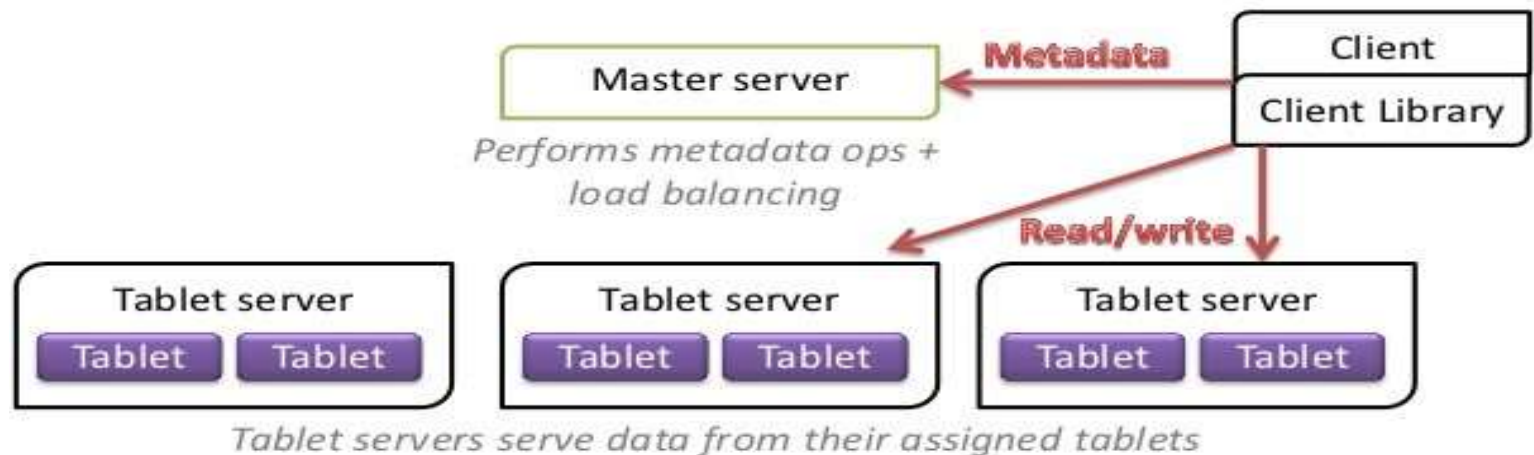
Tablet Server

- ▶ Can be **added** or **removed** **dynamically**.
- ▶ Each **manages** a set of tablets (typically 10-1000 tablets/server).
- ▶ Handles **read/write** requests to tablets.
- ▶ **Splits** tablets when too large.



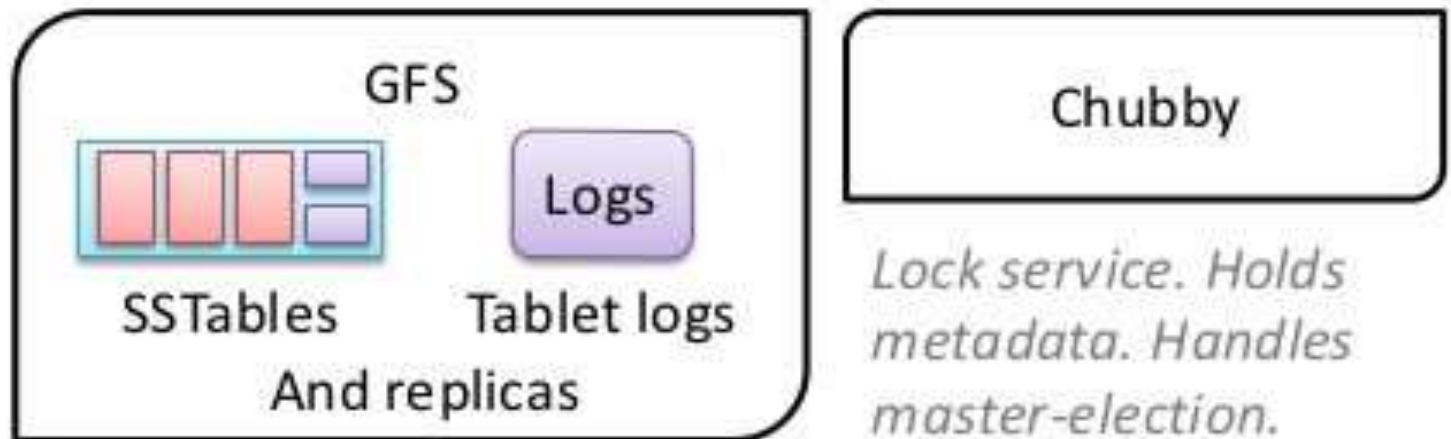
Client Library

- ▶ **Library** that is linked into every client.
- ▶ Client **data does not move** though the **master**.
- ▶ Clients communicate **directly** with **tablet servers** for **reads/writes**.



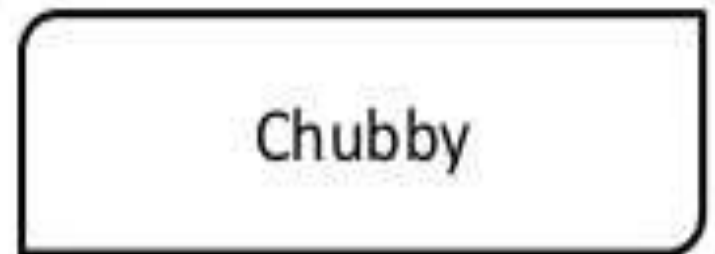
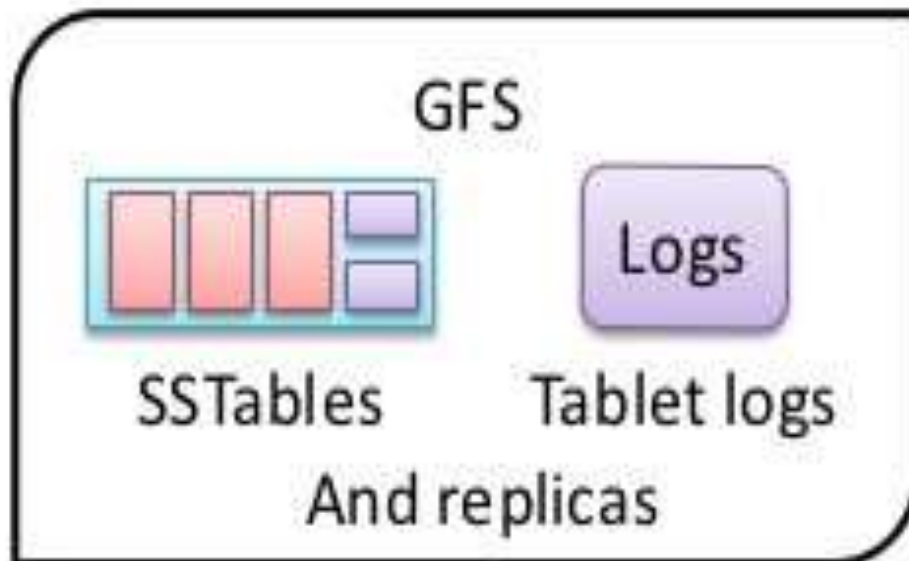
Building Blocks

- ▶ The building blocks for the BigTable are:
 - Google File System (GFS)
 - Chubby
 - SSTable



Google File System (GFS)

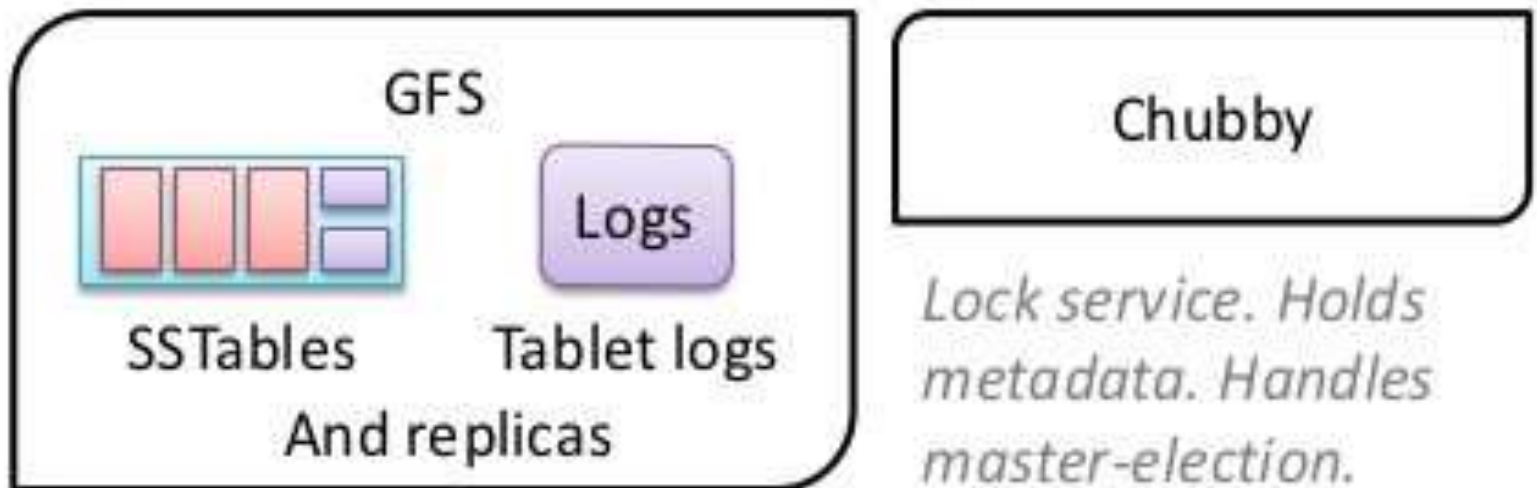
- ▶ Large-scale distributed file system.
- ▶ Store log and data files.



Lock service. Holds metadata. Handles master-election.

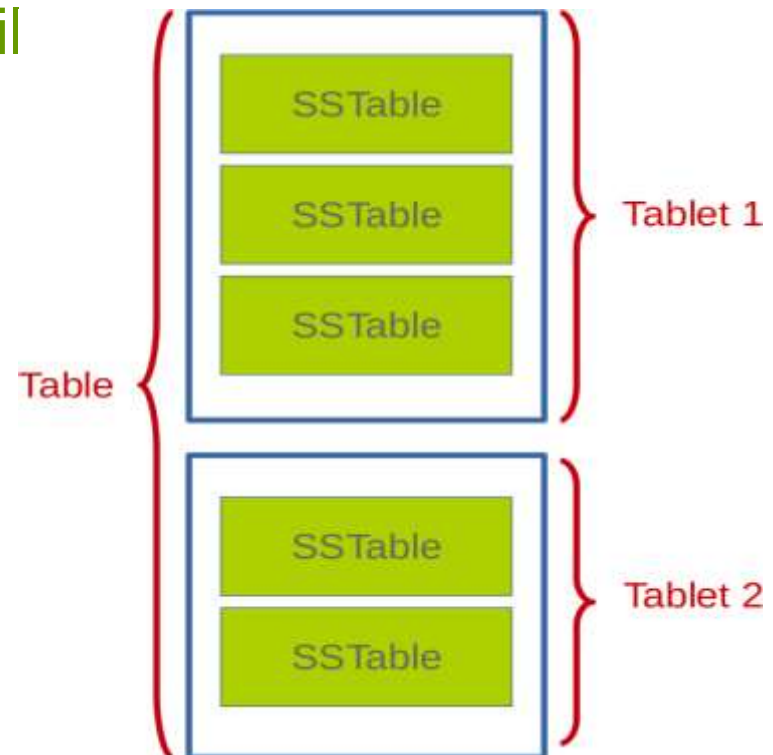
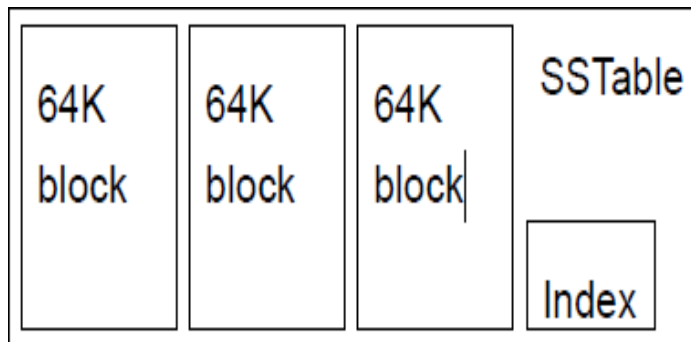
Chubby Lock Service

- ▶ Ensure there is only **one active master**.
- ▶ Store **bootstrap location** of BigTable data.
- ▶ **Discover** tablet servers.
- ▶ Store BigTable **schema** information and **access control lists**.



SSTable

- ▶ **SSTable** file format used internally to store BigTable data.
- ▶ Chunks of **data** plus a **block index**.
- ▶ **Immutable**, sorted file of **key-value** pairs.
- ▶ Each SSTable is stored in a **GFS** file



Data Partitioning (1/2)

- ▶ **Key/value**, where values are stored as **objects**.
- ▶ If size of data **exceeds the capacity** of a single machine: **partitioning**
- ▶ **Consistent hashing** for partitioning.



Data Partitioning (2/2)

- Consistent hashing.
- Hash both data and node ids using the same hash function in a same id space.
- $\text{partition} = \text{hash}(d) \bmod n$, d : data, n : the size of the id space

id space = $[0, 15]$, $n = 16$

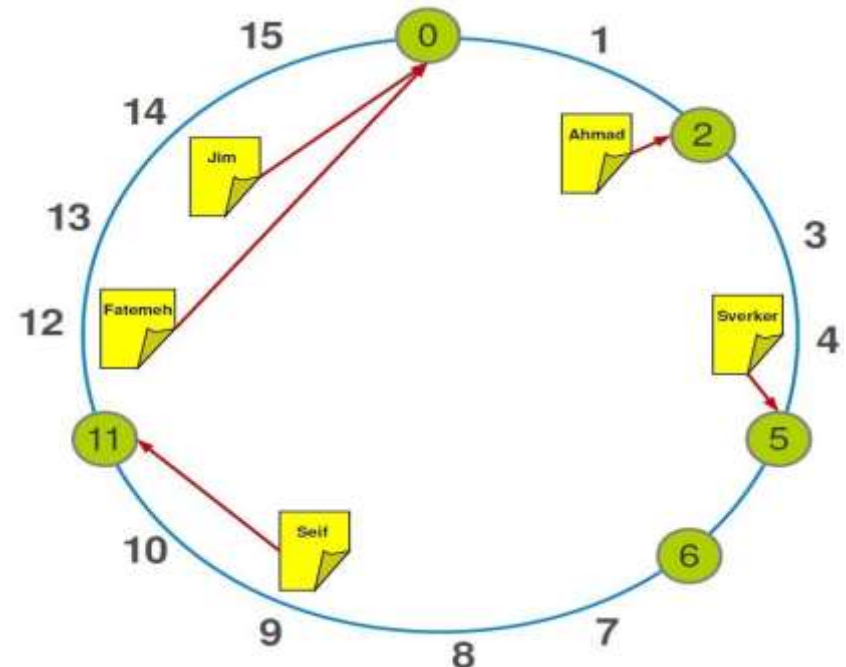
$\text{hash}(\text{"Fateme h"}) = 12$

$\text{hash}(\text{"Ahmad"}) = 2$

$\text{hash}(\text{"Seif"}) = 9$

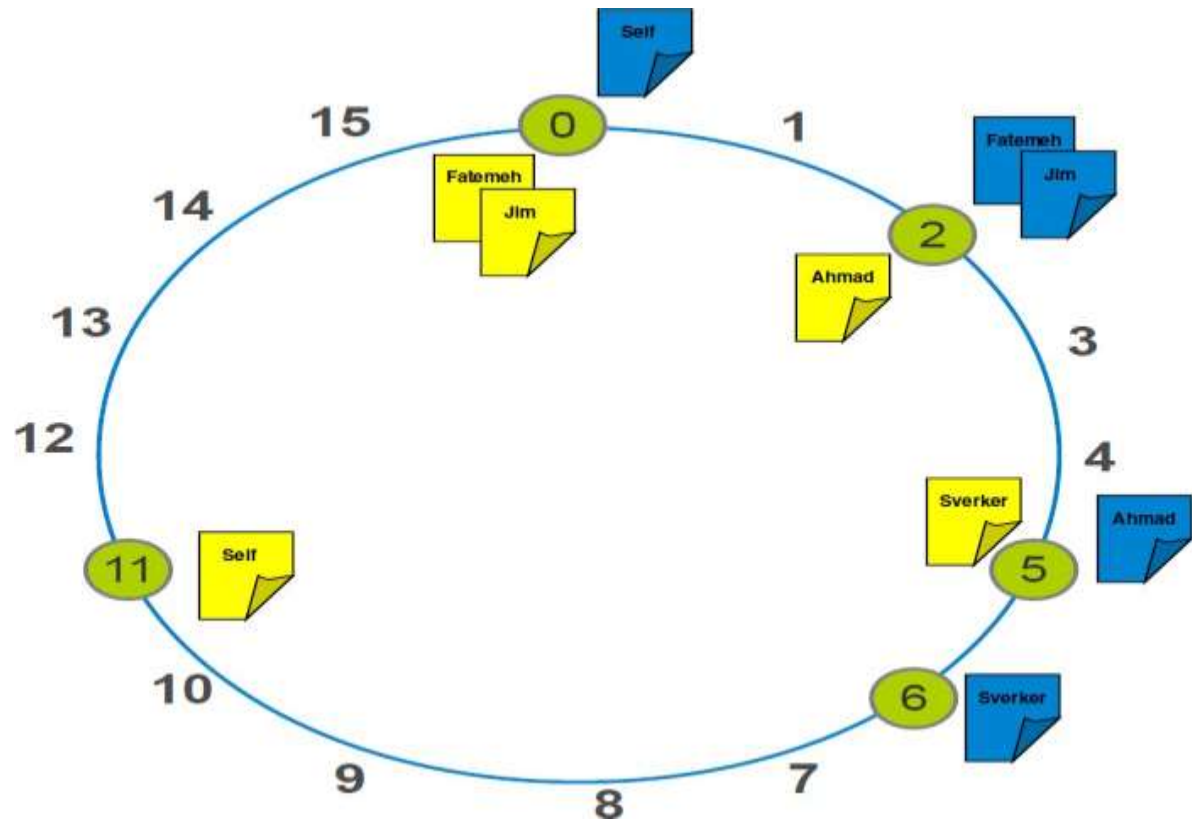
$\text{hash}(\text{"Jim"}) = 14$

$\text{hash}(\text{"Sverker"}) = 4$



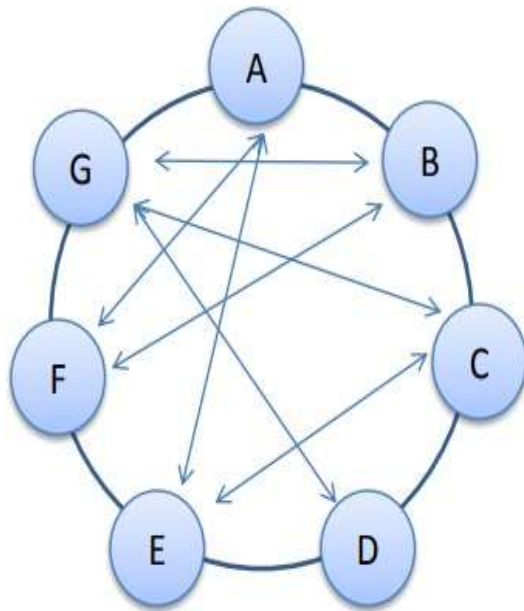
Replication

- ▶ To achieve high **availability** and **durability**, data should be **replicated** on multiple nodes.

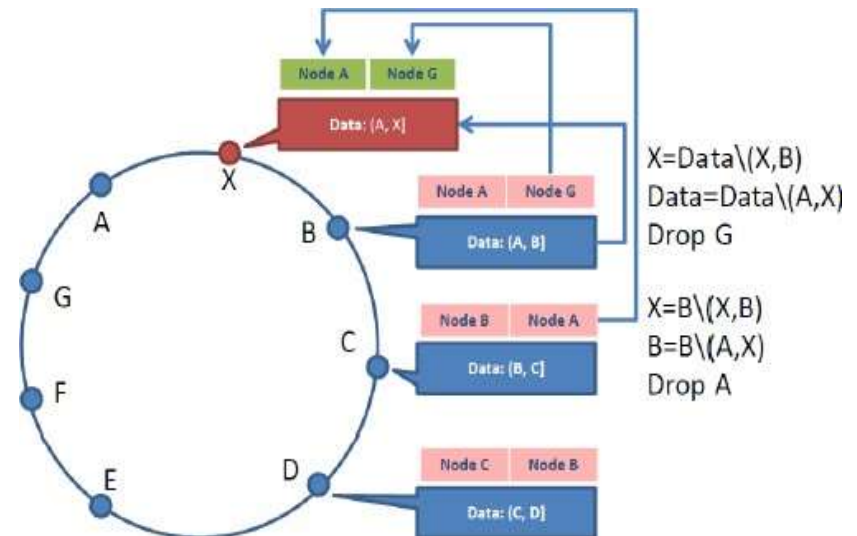


Adding and Removing Nodes

- Gossip-based mechanism: **periodically**, each node contacts **another randomly selected** node.



A → F
B → G
C → E
D → G
E → A
F → B
G → C



Cassandra Example

Create a keyspace called "test"

```
create keyspace test
```

```
with replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
```

Print the list of keyspaces

```
describe keyspaces;
```

Navigate to the "test" keyspace

```
use test
```

Create the "words" table in the "test" keyspace

```
create table words (word text, count int, primary key (word));
```

Insert a row

```
insert into words(word, count) values('hello', 5);
```

Look at the table

```
select * from words;
```

Performance

- There is no perfect NoSQL database
- Every database has its advantages and disadvantages
 - Depending on the type of tasks (and preferences) to accomplish
- NoSQL is a set of concepts, ideas, technologies, and software dealing with
 - Big data
 - Sparse un/semi-structured data
 - High horizontal scalability
 - Massive parallel processing
- Different applications, goals, targets, approaches need different NoSQL solutions